Information Processing Letters 39 (1991) 205–211
North-Holland

30 August 1991

Splitsort—An adaptive sorting algorithm

Christos Levcopoulos and Ola Petersson

Department of Computer Science, Lund University, Box 118, S-22100 Lund, Sweden

Communicated by T. Lengauer
Received 19 September 1990
Revised 4 March 1991

Abstract

Levcopoulos, C. and O. Petersson, Splitsort—An adaptive sorting algorith, Information Processing Letters 39 (1991) 205–211.

We present a new sorting algorithm, called Splitsort, which adapts to existing order within the input sequence. The algorithm is optimal with respect to several known measures of presortedness, including the number of inversions, for which no such simple and space efficient algorithm was known before. The amount of extra space needed is only n pointers. Splitsort uses a simple data structure and is easy to code. In the worst case Splitsort performs $2.5n\log_2 n$ comparisons, but if the input is presorted according to some of the measures it completes the sorting task considerably faster. We also show how a variant of the algorithm can be implemented to run in-place.

Keywords: Analysis of algorithms, design of algorithms, sorting, presortedness, inversions

1. Introduction

It is well known that, in a comparison-based model of computation, $\Omega(n \log n)$ time is necessary to sort n elements in both the worst case and the average case [11]. This result relies on the assumption that the elements are randomly permuted. In many applications this is not the case, but the sequence to be sorted is already partially ordered (*presorted*). Most $O(n \log n)$ time algorithms, e.g., Heapsort and Mergesort, do not take existing order within their input into account.

A *measure of presortedness* is a nonnegative integer function on a sequence X that reflects how much X differs from the sorted permutation of X. Examples of presortedness measures include the number of inversions and the number of runs. Although earlier considered by Guibas et al. [5], the term presortedness was coined by Mehlhorn [10], who used the number of inversions in the sequence as a measure. Mehlhorn [10,11] also gave an algorithm, A-Sort, which is adaptive with respect to this measure. Intuitively, an algorithm is *adaptive* with respect to a measure of presortedness if it sorts all permutations, but performs particularly well on permutations that have a high degree of presortedness, without knowing the value of the measure beforehand. Mannila [9] formalized the concept of presortedness and gave an algorithm which is adaptive with respect to several measures. Recently, Estivill-Castro and Wood [4], Carlsson, Levcopoulos, and Petersson [1], Levcopoulos and Petersson [7,8], and Skiena [12] have considered other measures. Cook and Kim [2], Dromey [3], and Wainwright [13], among others, have studied the problem of designing adaptive sorting algorithms empirically.

This paper describes a new adaptive sorting algorithm, called Splitsort. Splitsort is optimal with respect to *Inv*, the number of inversions in a sequence, as well as to *Rem*, the minimum number of elements whose removal leaves a sorted sequence, and *Exc*, the minimum number of exchanges needed to sort a sequence. Here, optimal-

0020-0190/91/$03.50 © 1991 – Elsevier Science Publishers B.V. All rights reserved

ity with respect to a measure means maximum adaptation (in an asymptotic sense). In the worst case Splitsort performs $2.5n \log n$ comparisons. The main contribution of the paper is the low constant of proportionality and the simplicity of Splitsort compared to all other known algorithms which are optimal with respect to the number of inversions. Splitsort is also space efficient: a straightforward implementation uses $n + O(\log n)$ pointers in addition to the input array. Further, the algorithm is easy to code.

The main idea in Splitsort comes from Cook and Kim [2]. They provided a linear time method for removing $\Theta(Rem(X))$ elements from a sequence X, such that what is left in X is a sorted sequence. What Splitsort does is splitting the extracted elements into two subsequences of equal length so that the total number of inversions decreases by a constant factor. These two subsequences are then sorted recursively and finally merged with what was left of the original sequence, yielding the sorted output.

We also provide an in-place implementation of Cook and Kim's method. Together with Heapsort and in-place merging, this yields an in-place sorting algorithm which is optimal with respect to *Rem*.

The remainder of the paper is organized as follows. In Section 2 we state some preliminary definitions and basic results. In Section 3 Splitsort is presented, and in Section 4 its behavior with respect to *Inv* and *Rem* is analyzed. In Section 5 we show how Cook and Kim's method can be implemented in-place. Finally, in Section 6 we summarize the results.

2. Preliminaries

Let $X = \langle x_1, \ldots, x_n \rangle$ be a *sequence* of n elements x_i from some totally ordered set. Further, let $|X|$ denote the *length* of X, $\|S\|$ the *cardinality* of a set S, and $\log x = \log_2(\max\{2, x\})$. A sequence obtained by deleting zero or more elements from X is called a *subsequence* of X.

For a sequence X of length n, the two well-known measures of presortedness we consider are formally defined as

$$Inv(X) = \|\{(i, j) \mid 1 \leqslant i < j \leqslant n \text{ and } x_i > x_j\}\|,$$

$$Rem(X) = n - \max\{k \mid X \text{ has an ascending subsequence of length } k\}.$$

The concept of an optimal algorithm with respect to a measure of presortedness was given in a general form by Mannila [9]:

Definition 1. Let M be a measure of presortedness, and S a comparison-based sorting algorithm that uses $T_S(X)$ steps to sort a sequence X. We say that S is *M-optimal*, or *optimal with respect to* M, if there is a constant $c > 0$, such that for every sequence X, we have

$$T_S(X) \leqslant c \cdot (|X| + \log \|below(X, M)\|),$$

where $below(X, M) = \{\pi \mid \pi \text{ is a permutation of } \{1, \ldots, |X|\} \text{ and } M(\pi(X)) \leqslant M(X)\}$.

Giving a lower bound for algorithms with respect to a measure of presortedness thus means to bound the cardinality of the *below*-set from below. Let X be a sequence of length n. For *Inv*, Guibas et al. [5] proved

$$n + \log \|below(X, Inv)\| = \Omega\left(n \cdot \log\left(\frac{Inv(X)}{n}\right)\right), \quad (1)$$

and for *Rem*, Mannila [9] proved

$$n + \log \|below(X, Rem)\| = \Omega(n + Rem(X) \cdot \log Rem(X)). \quad (2)$$

3. The algorithm

As mentioned above, Splitsort applies the linear time method of Cook and Kim [2] for removing $\Theta(Rem(X))$ elements from X, such that what remains in X is an ascending subsequence of X. This can be done as follows. We scan the sequence forward while keeping track of an *upsequence* (ascending subsequence) in X. The upsequence,

which initially contains x_1, is updated by comparing its *head* (last element) with the next element encountered during the scan, x_i. If x_i is smaller, both x_i and the head are removed, and the former head becomes the new head. Otherwise, no element is removed and x_i is made the new head. It is clear that, when finished, what remains in X is an ascending subsequence of the original sequence. To derive an upper bound on the number of removed elements, consider a removed pair. Since the pair is in wrong order both elements cannot possibly belong to a longest ascending subsequence of X. Consequently, by the definition of *Rem*, at least one of them contributes to $Rem(X)$. Hence, the number of removed elements is at most $2 \cdot Rem(X)$. Since in each step a new element is compared to the head of the upsequence, it follows that linear time suffices. Further, compaction of the removed elements as well as the upsequence is also done in linear time.

Next, the removed elements are sorted separately and merged with the upsequence. Rather than just viewing these elements as a random sequence, however, Splitsort exploits the fact that they were removed pairwise, in unordered pairs. The removed elements are split into two subsequences X_G and X_S. X_G contains all removed elements that were greater in their respective pairs and X_S those that were smaller. Moreover, the splitting is performed in a stable way, i.e., the order in which the elements appear in X_G and X_S is the same as in which they appeared in X. The reason for doing the splitting this way becomes evident in the analysis. X_G and X_S are then sorted recursively and finally merged with the upsequence that was left in X.

We are now ready for a less informal description of Splitsort. Let $Split(X, X_G, X_S)$ be a procedure that performs the separation outlined above, and let $Merge(X, X_G, X_S)$ be a procedure that merges three sorted sequences.

```
procedure Splitsort(var X: sequence);
  Split(X, X_G, X_S);
  if |X_S| > 1 then
    Splitsort(X_G);
    Splitsort(X_S);
  endif;
  Merge(X, X_G, X_S);
end;
```

4. Upper bounds

We establish upper bounds on the number of comparisons performed by Splitsort in terms of *Rem* and *Inv*. First, however, we analyze its worst-case behavior independently of measures of presortedness.

Lemma 2. *Splitsort spends at most* $\min\{2n + 2|X_S|, 3n - |X_S|\}$ *comparisons on each level of recursion.*

Proof. *Split* performs at most n comparisons, because each comparison made involves at least one new element.

When merging we always start by merging X_S with the shortest of the other two, and then merge the remaining one with the resulting sequence. If X_G is shorter, the number of comparisons is at most

$$(|X_S| + |X_G|) + n = n + 2|X_S|,$$

because $|X_G| = |X_S|$. Otherwise, we spend

$$(|X_S| + n - |X_S| - |X_G|) + n = 2n - |X_S|$$

comparisons.

By adding the costs for splitting and merging, the lemma follows. □

Lemma 3. *Splitsort sorts a sequence X of length n in* $2.5n \log n + n/2$ *comparisons.*

Proof. By the preceding lemma, the number of comparisons spent by Splitsort is given by

$$C(n) \leqslant \begin{cases} 0 & \text{if } n \leqslant 1, \\ \max\{2 \cdot C(|X_S|) \\ \quad + \min\{2n + 2|X_S|, 3n - |X_S|\}\} \\ \quad \text{otherwise,} \end{cases}$$

where the maximum is taken over all possible values of $|X_S|$, that is, $0 \leqslant |X_S| \leqslant \lfloor n/2 \rfloor$.

We prove that $C(n) \leqslant 2.5n \log n + n/2$, by induction on n. It is certainly true for $n = 1$. Assume that it holds for every value less than n. Applying the induction hypothesis for $n > 1$ gives

$$\begin{aligned} C(n) &\leqslant \max\{5|X_S| \log |X_S| + |X_S| \\ &\qquad + \min\{2n + 2|X_S|, 3n - |X_S|\}\} \\ &= \max\{5|X_S| \log |X_S| \\ &\qquad + \min\{2n + 3|X_S|, 3n\}\}. \end{aligned}$$

Replacing $|X_S|$ by its maximum value in the first term gives

$$\begin{aligned} C(n) &\leqslant \max\{2.5n \log(n/2) \\ &\qquad + \min\{2n + 3|X_S|, 3n\}\} \\ &= \max\{2.5n \log n \\ &\qquad + \min\{3|X_S| - n/2, n/2\}\}. \end{aligned}$$

The second term is maximized when $3|X_S| - n/2 = n/2$, which is attained for $|X_S| = n/3$. Then $\min\{3|X_S| - n/2, n/2\} = n/2$. Hence,

$$C(n) \leqslant 2.5n \log n + n/2,$$

which completes the induction and concludes the proof. □

Lemma 4. *Splitsort sorts a sequence X of length n with $Rem(X) = R$ in $5R \log R + O(n)$ comparisons.*

Proof. Recall that the number of elements in each of X_G and X_S after the first *Split* is at most $Rem(X)$. The claim follows if we apply Lemma 3 to each of these subsequences and add a linear term for splitting and merging on the first level of recursion. □

When establishing an upper bound on Splitsort in terms of *Inv*, the key observation is that the number of inversions decreases by a constant factor on each level of recursion. More precisely,

Lemma 5. $Inv(X_G) + Inv(X_S) \leqslant (2/3) \cdot Inv(X)$.

Proof. Consider an inversion (i, j) in X_G, i.e., $i < j$ and $x_i > x_j$. [1] Since x_j is in X_G, and as the elements were removed pairwise, there must be an element x_l in X_S, such that $j < l$ and $x_j > x_l$. Then (i, l) is an inversion in X, which does not appear in any of X_G and X_S. Therefore,

$$Inv(X) \geqslant 2 \cdot Inv(X_G) + Inv(X_S).$$

By considering an inversion in X_S, an analogous argument gives

$$Inv(X) \geqslant Inv(X_G) + 2 \cdot Inv(X_S).$$

Hence,

$$\begin{aligned} Inv(X) &\geqslant Inv(X_G) + Inv(X_S) \\ &= \max\{Inv(X_G), Inv(X_S)\} \\ &\geqslant Inv(X_G) + Inv(X_S) \\ &\qquad + \frac{Inv(X_G) + Inv(X_S)}{2}, \end{aligned}$$

from which the claim follows. □

Lemma 6. *For any sequence X, $Rem(X) \leqslant Inv(X)$.*

Proof. Let X_1 be any longest upsequence in X, and let x_j be an element in X that is not in X_1. Denote by x_i the greatest element in X_1 that precedes x_j in X, and similarly, let x_k be the smallest element in X_1 that follows x_j in X. Note that if x_j is the first or last element in X, both x_i and x_k do not exist, but at least one of them does. If $x_i \leqslant x_j \leqslant x_k$, then x_j can be added to X_1, contradicting the assumptions. Hence, at least one of $x_i > x_j$ and $x_j > x_k$ holds, that is, at least one of (i, j) and (j, k) is an inversion. We conclude that each element x_j contributing to $Rem(X)$ causes at least one inversion. □

Lemma 7. *Splitsort sorts a sequence X of length n with $Inv(X) = I$ in $4.6n \log(I/n) + O(n)$ comparisons.*

Proof. The analysis is divided into two phases: first, we take care of the highest levels of the recursion, when the number of inversions is greater than $n/4$; second, we examine what happens on the lower levels.

Let $Inv_i(X)$ denote the total number of inversions in the subsequences on the ith level of

[1] For simplicity, let i and j be the indices in X rather than in X_G. Since *Split* is stable, this makes no difference.

recursion; e.g., $Inv_1(X) = Inv(X_G) + Inv(X_S)$. On the lth level of recursion, where

$$l = \log(Inv(X)/n)/\log(3/2) + 4,$$

we have

$$Inv_l(X) \leqslant \left(\tfrac{2}{3}\right)^l \cdot Inv(X) \leqslant \frac{n}{4}, \qquad (3)$$

by Lemma 5. By Lemma 2, the number of comparisons performed on each level of recursion is given by

$$\min\{2n + 2|X_S|, 3n - |X_S|\} \leqslant 8n/3.$$

Hence, the total number of comparisons performed on the first l levels of recursion is at most

$$\frac{8n}{3} \cdot l \leqslant \frac{8n}{3} \cdot \frac{\log(Inv(X)/n)}{\log(3/2)} + O(n)$$

$$< 4.6n\log\left(\frac{Inv(X)}{n}\right) + O(n).$$

The remaining levels of recursion will take linear time altogether. To verify this, let $|(X, i)|$ denote the total length of the X_G's and X_S's on the ith level of recursion; e.g., $|(X, 1)| = |X_G| + |X_S|$. As $Rem(X) \leqslant Inv(X)$, by Lemma 6, and since at most $2 \cdot Rem(X)$ elements take part in recursive calls, we have

$$|(X, i)| \leqslant 2 \cdot Inv_{i-1}(X).$$

Applying Lemma 5 gives

$$|(X, l+i)| \leqslant 2 \cdot Inv_{l+i-1}(X)$$

$$\leqslant 2 \cdot \left(\frac{2}{3}\right)^{i-1} \cdot \left(\frac{2}{3}\right)^l \cdot Inv(X)$$

$$\leqslant \left(\frac{2}{3}\right)^{i-1} \cdot \frac{n}{2},$$

by (3). Hence, $|(X, l+1)| \leqslant n/2$, $|(X, l+2)| \leqslant n/3$, and so on. Therefore, the remaining lower levels take time

$$T(n/2) \leqslant \begin{cases} O(1) & \text{if } n \leqslant 1, \\ T(n/3) + O(n) & \text{otherwise,} \end{cases}$$

which is linear. We conclude that Splitsort uses $4.6n\log(Inv(X)/n)) + O(n)$ comparisons. □

Before we state the main result of the paper, consider the measure *Exc*, defined as

$Exc(X)$ = the minimum number of exchanges needed to bring X into ascending order.

Carlsson, Levcopoulos, and Petersson [1] proved the following relation between *Exc* and *Rem*:

Lemma 8. *Every Rem-optimal sorting algorithm is also Exc-optimal.*

To summarize, we combine Lemmas 3, 4, 7 and 8, the lower bounds given by (1) and (2), and Definition 1 in the following:

Theorem 9. *Splitsort sorts a sequence X of length n with $Rem(X) = R$ and $Inv(X) = I$ in at most*

$$O(n) + \min\{2.5n \log n, 5R \log R, 4.6n \log(I/n)\}$$

comparisons, which is optimal with respect to Rem, Exc, and Inv.

Let us briefly comment on the implementation and space complexity of Splitsort. If both *Split* and *Merge* return their outputs in the same memory area as the inputs are given, linear space suffices. But how large is the implicit constant factor?

A straightforward implementation of *Merge* uses n extra element locations. Using standard techniques it can even be implemented with only n extra pointers, without increasing the number of comparisons performed.

Implementing *Split* is not that obvious. To see why, note that during the scan, the upsequence is treated as a stack with constant time operations. Hence, the upsequence needs to be linked together in some way. Moreover, the elements in X_G, as opposed to those in X_S, are not necessarily removed in the correct order. For example, if X is a concatenation of two sorted sequences X_1 and X_2 of equal lengths, where every element in X_1 is greater than every element in X_2, the elements in X_G are removed in reverse order, while the ones in X_S are removed in the desired order.

One way of implementing *Split* is to make use

of an auxiliary array of n pointers. The elements in the upsequence are linked together using these pointers, and the other entries are used for indicating which of X_G and X_S the corresponding elements belong to. After the first scan, we scan the auxiliary array, writing into each entry the final position of the corresponding element. Finally, we rearrange the input array according to the addresses in the auxiliary array. (The details are left to the interested reader.) Hence, *Split* can be implemented using only n extra pointers. Since the same amount of space is sufficient for *Merge*, we conclude that Splitsort requires at most n pointers plus O(log n) for the recursion in addition to the input array.

5. An in-place algorithm

Recall that, as was shown in Section 3, the number of elements removed by Cook and Kim's method is at most $2 \cdot Rem(X)$. The reader might have observed that by simply applying a worst-case optimal sorting algorithm to sort the removed elements, and finally a merge with the upsequence we end up with an algorithm running in time

$$O(n + Rem(X) \log Rem(X)),$$

which is *Rem*-optimal. [2]

In this section we show how to implement Cook and Kim's method using only constant extra space, without increasing the time complexity. To achieve this space efficiency, however, we loose the stability of the separation. The idea is to keep the upsequence in the first portion of the array X during the scan. When the scan is finished the extracted elements will occupy the last portion of X.

Assume inductively that when x_i is encountered, the first l positions of X are hold by the upsequence and the following $i-1-l$ positions contain the elements removed so far. Further, assume that the upsequence is stored in order, i.e., $x_1 \leqslant x_2 \leqslant \cdots \leqslant x_l$. When comparing x_i and x_l we follow the strategy: if x_i is greater, swap x_i and x_{l+1} and increment l by one; otherwise, just decrement l by one. It is obvious that the outlined method performs the separation of X into the upsequence in order in the first portion of the array followed by the elements belonging to unordered pairs in the remaining portion. Further, only constant extra space is used.

Theorem 10. *There is an in-place sorting algorithm which is Rem-optimal.*

Proof. Consider the following algorithm:

1. Perform the above outlined removals of the unordered pairs.
2. Apply Heapsort to sort the extracted elements.
3. Merge the two resulting sorted sequences.

The first step runs in linear time and constant extra space by the preceding discussion. Since the number of extracted elements is at most $2 \cdot Rem(X)$, step two runs in

$$O(Rem(X) \log Rem(X))$$

time because Heapsort is worst-case optimal. Also, only constant extra space is used by Heapsort. By using an in-place merge, e.g., the one by Huang and Langston [6], in the third step, it runs in linear time and constant extra space. To sum up, the time consumed by our algorithm is

$$O(n + Rem(X) \log Rem(X))$$

which is *Rem*-optimal, by the lower bound in Section 2, and the amount of extra space needed is only constant. □

We close this section by emphasizing the appropriateness of Cook and Kim's method as a preprocessing phase in any adaptive sorting algorithm which is not optimal with respect to *Rem* in its original setting. Consider a measure of presortedness M, which has the property that if Y is a subsequence of the sequence X, then $M(Y) \leqslant M(X)$. [3] Assume that there is a sorting algorithm

[2] In their paper, Cook and Kim apply Quickersort or Straight Insertion Sort, which are not worst-case optimal, to sort the extracted elements to achieve good empirical results rather than theoretical.

[3] According to Mannila's definition [9] this property is a necessary condition for a function to qualify as a measure of presortedness.

M-sort which is *M*-optimal but not *Rem*-optimal. A way to make *M*-sort *Rem*-optimal as well is as follows. Start by applying Cook and Kim's method on the input sequence; however, make it stably, i.e., remove all elements belonging to the unordered pairs and put them in one new sequence in which the elements' mutual order in unaltered. Second, sort the removed elements by *M*-sort, and finally, merge them with the remaining upsequence. Clearly, the resulting algorithm is both *M*-optimal and *Rem*-optimal.

6. Conclusion

We have presented a new adaptive sorting algorithm, named Splitsort. Splitsort was shown to be optimal with respect to the measures *Inv*, *Rem*, and *Exc*. Basically, Splitsort scans the input sequence and removes pairs of elements that are out of order. These elements are split into two subsequences which are sorted recursively and finally merged with the elements left in the original sequence.

The main contribution of the paper is the simplicity and space efficiency of Splitsort compared to other *Inv*-optimal algorithms. The most well-known *Inv*-optimal algorithm is probably A-Sort of Mehlhorn [11], which is an insertion sort algorithm where the already inserted elements are implemented by an (a, b)-tree. Local Insertion Sort of Mannila [9] uses an even more complicated data structure, a finger tree. This, however, yields optimality with respect to other measures of presortedness as well. Recently, Levcopoulos and Petersson [7] provided a variant of Heapsort, Adaptive Heapsort, which is *Inv*-optimal. Their algorithm is simpler than both Mehlhorn's and Mannila's but comprises a linear preprocessing phase which makes it more complicated to implement than Splitsort.

We also pointed out that Cook and Kim's method can be implemented to run in-place. As a consequence a simple in-place *Rem*-optimal sorting algorithm was obtained.

Acknowledgment

We would like to thank Svante Carlsson for interesting discussions, and the anonymous referees for helpful comments.

References

[1] S. Carlsson, C. Levcopoulos and O. Petersson, Sublinear merging and Natural Mergesort, in: *Proc. SIGAL Internat. Symp. on Algorithms*, Lecture Notes in Computer Science **450** (Springer, Berlin, 1990) 251–260.

[2] C.R. Cook and D.J. Kim, Best sorting algorithms for nearly sorted lists, *Comm. ACM* **23** (11) (1980) 620–624.

[3] G.R. Dromey, Exploiting partial order with Quicksort, *Software — Practice and Experience* **14** (6) (1984) 509–518.

[4] V. Estivill-Castro and D. Wood, A new measure of presortedness, *Inform. and Comput.* **83** (1) (1989) 111–119.

[5] L.J. Guibas, E.M. McCreight, M.F. Plass and J.R. Roberts, A new representation of linear lists, in: *Proc. 9th Ann. ACM Symp. on Theory of Computing* (1977) 49–60.

[6] B-C. Huang and M.A. Langston, Practical in-place merging, *Comm. ACM* **31** (3) (1988) 348–352.

[7] C. Levcopoulos and O. Petersson, Heapsort—adapted for presorted files, in: *Proc. 1989 Workshop on Algorithms and Data Structures*, Lecture Notes in Computer Science **382** (Springer, Berlin, 1989) 499–509.

[8] C. Levcopoulos and O. Petersson, Sorting shuffled monotone sequences, in: *Proc. 2nd Scandinavian Workshop on Algorithm Theory*, Lecture Notes in Computer Science **447** (Springer, Berlin, 1990) 181–191.

[9] H. Mannila, Measures of presortedness and optimal sorting algorithms, *IEEE Trans. Comput.* **34** (4) (1985) 318–325.

[10] K. Mehlhorn, Sorting presorted files, in: *Proc. 4th GI Conf. on Theoretical Computer Science*, Lecture Notes in Computer Science **67** (Springer, Berlin, 1979) 199–212.

[11] K. Mehlhorn, *Data Structures and Algorithms, Vol. 1: Sorting and Searching* (Springer, Berlin/Heidelberg, 1984).

[12] S.S. Skiena, Encroaching lists as a measure of presortedness, *BIT* **28** (4) (1988) 775–784.

[13] R.L. Wainwright, A class of sorting algorithms based on Quicksort, *Comm. ACM* **28** (4) (1985) 396–402.